EXERCICE 03 / 4

LA SECURITE

JWT · Authentification · Validation · Swagger

Difficulte : ★★★☆ 24 heures

Contexte pedagogique

Une API publique sans securite est inutilisable en production. Cet exercice introduit l'authentification par JWT (JSON Web Token) — le standard de l'industrie pour les API REST. L'apprenant protege les routes sensibles, valide rigoureusement les entrees, et documente l'API avec Swagger pour que les autres developpeurs puissent l'utiliser.

Competences ciblees

- Hachage de mots de passe avec bcrypt (jamais stocker en clair)
- JWT : structure (header.payload.signature), generation, verification
- Middleware d'authentification : verifier le token dans Authorization Bearer
- Routes publiques vs protegees : register, login public / sensors prive
- Validation avancee avec express-validator
- Variables d'environnement : JWT_SECRET, JWT_EXPIRATION
- Documentation Swagger/OpenAPI avec swagger-jsdoc et swagger-ui-express
- Tester les routes protegees dans Postman avec le token JWT

CONFIGURATION MOODLE — Parametres de l'activite

```
Type d'activite   : Devoir (Assignment)
Titre             : EX03 — La Securite : JWT & Authentification
Prerequis         : EX02 valide >= 6/10
Delai             : 24 heures apres deblocage
Rendu             : Lien GitHub — branche ex03-auth + URL Swagger
accessible
Note max          : 10 points
Note de passage   : 6/10 pour debloquer EX04
Ressources        : Video JWT explique (15min) + Video Swagger (15min) +
Fiche bcrypt
```

1 Authentification — Register et Login

Creer les routes d'inscription et de connexion avec hachage bcrypt et generation de JWT.

```
// Terminal + JavaScript — Installation et config auth
npm install jsonwebtoken bcryptjs express-validator

// src/routes/auth.js
const router    = require('express').Router();
const bcrypt    = require('bcryptjs');
const jwt       = require('jsonwebtoken');
```

```
const { body, validationResult } = require('express-validator');

// Base de donnees temporaire (remplacee par PostgreSQL en EX04)
const users = [];

// POST /api/auth/register
router.post('/register',
  body('email').isEmail().normalizeEmail(),
  body('password').isLength({ min: 6 }),
  body('name').trim().isLength({ min: 2 }),
  async (req, res, next) => {
    const errors = validationResult(req);
    if (!errors.isEmpty())
      return res.status(400).json({ success: false, errors: errors.array()
});

    try {
      const { email, password, name } = req.body;
      if (users.find(u => u.email === email))
        return res.status(409).json({ success: false, error: 'Email deja
utilise' });

      const hashed = await bcrypt.hash(password, 12); // 12 rounds de
salage
      const user   = { id: Date.now(), name, email, password: hashed };
      users.push(user);

      res.status(201).json({ success: true, message: 'Compte cree avec
succes' });
    } catch (err) { next(err); }
  }
);
```

- bcrypt.hash(password, 12) : le chiffre 12 est le nombre de rounds de salage — plus c'est eleve, plus c'est lent et securise
- Ne JAMAIS retourner le mot de passe hache dans la reponse — meme hache
- 409 Conflict : code HTTP correct quand une ressource existe deja (email duplique)

2 Generation et verification du JWT

```
// JavaScript — Login et generation du token JWT
// POST /api/auth/login
router.post('/login',
  body('email').isEmail(),
  body('password').notEmpty(),
  async (req, res, next) => {
    const errors = validationResult(req);
    if (!errors.isEmpty())
      return res.status(400).json({ errors: errors.array() });

    try {
      const { email, password } = req.body;
      const user = users.find(u => u.email === email);
```

```
      if (!user || !(await bcrypt.compare(password, user.password)))
        return res.status(401).json({ success: false, error: 'Identifiants
incorrects' });

      // Generer le JWT
      const token = jwt.sign(
        { id: user.id, email: user.email, name: user.name }, // Payload
        process.env.JWT_SECRET,                              // Cle
secrete
        { expiresIn: process.env.JWT_EXPIRATION || '24h' }   // Expiration
      );

      res.status(200).json({ success: true, token, user: { id: user.id,
name: user.name, email: user.email } });
    } catch (err) { next(err); }
  }
);

// src/middleware/auth.js — Verifier le token sur les routes protegees
const jwt = require('jsonwebtoken');

const authenticate = (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer '))
    return res.status(401).json({ success: false, error: 'Token manquant'
});

  const token = authHeader.split(' ')[1];
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (err) {
    res.status(401).json({ success: false, error: 'Token invalide ou
expire' });
  }
};

module.exports = authenticate;
```

- jwt.sign() cree le token. jwt.verify() le valide et retourne le payload decode
- Le token est envoye dans le header Authorization: Bearer <token> — jamais dans l'URL
- req.user = payload decode : les routes suivantes connaissent l'utilisateur connecte

3 Proteger les routes et validation express-validator

```
// JavaScript — Routes protegees et validation complete
// Appliquer le middleware authenticate sur les routes protegees
router.get('/',     authenticate, controller.getAll);
router.post('/',    authenticate, validateSensor, controller.create);
router.put('/:id', authenticate, validateSensor, controller.update);
router.delete('/:id', authenticate, controller.remove);
```

```
// Validation avancee avec express-validator
const validateSensor = [
  body('name')
    .trim()
    .isLength({ min: 2, max: 50 })
    .withMessage('Le nom doit avoir entre 2 et 50 caracteres'),
  body('type')
    .isIn(['temperature', 'humidity', 'light', 'pressure'])
    .withMessage('Type invalide'),
  body('value')
    .optional()
    .isFloat({ min: -100, max: 10000 })
    .withMessage('Valeur numerique entre -100 et 10000'),
  body('unit')
    .optional()
    .isString()
    .trim(),
  (req, res, next) => {
    const errors = validationResult(req);
    if (!errors.isEmpty())
      return res.status(400).json({ success: false, errors: errors.array()
});
    next();
  }
];
```

- Le tableau [authenticate, validateSensor, controller.create] : les middleware s'executent dans l'ordre
- body('value').optional() : la validation ne s'applique que si le champ est present

4 Documentation Swagger

Generer automatiquement une documentation interactive de l'API avec Swagger UI.

```
// Terminal + JavaScript — Configuration Swagger
npm install swagger-jsdoc swagger-ui-express

// src/config/swagger.js
const swaggerJsdoc = require('swagger-jsdoc');

const options = {
  definition: {
    openapi: '3.0.0',
    info: {
      title:   'CIR IoT API',
      version: '1.0.0',
      description: 'API REST pour la gestion des capteurs IoT — Centre
Ivoirien de Robotique',
    },
    components: {
      securitySchemes: {
        bearerAuth: { type: 'http', scheme: 'bearer', bearerFormat: 'JWT'
}
      }
    }
```

```
  },
  apis: ['./src/routes/*.js'], // Lire les commentaires JSDoc dans les
routes
};

module.exports = swaggerJsdoc(options);

// Dans app.js :
const swaggerUi   = require('swagger-ui-express');
const swaggerSpec  = require('./config/swagger');
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));
// Accessible sur : http://localhost:3000/api-docs

// Commenter une route pour Swagger :
/**
 * @swagger
 * /api/sensors:
 *   get:
 *     summary: Retourne tous les capteurs IoT
 *     security: [{ bearerAuth: [] }]
 *     responses:
 *       200:
 *         description: Liste des capteurs
 */
router.get('/', authenticate, controller.getAll);
```

- Swagger UI genere une page web interactive sur /api-docs — on peut tester l'API directement dans le navigateur
- Ajouter le commentaire JSDoc sur au moins 5 routes (GET all, GET id, POST, PUT, DELETE)